Technical Report

pyedger: a Python implementation of edgeR for differential expression analysis of count data

Zehua Zeng,[1,2,*] **Meng Xu,**[3] **and Hongwu Du**[4,5,*]
[1]Department of Genetics, Stanford University, Stanford, CA 94305, USA.
[2]OmicVerse, Inc., San Francisco, CA, USA.
[3]Hubei Key Laboratory of Agricultural Bioinformatics, College of Informatics, Huazhong Agricultural University, Wuhan 430070, China.
[4]School of Chemistry and Biological Engineering, University of Science and Technology Beijing, Beijing 100083, China.
[5]Daxing Research Institute, University of Science and Technology Beijing, Beijing 100083, China.
*Correspondence: starlitnightly@gmail.com, hongwudu@ustb.edu.cn

Summary

pyedger is a Python implementation of the Bioconductor package edgeR for differential expression analysis of count data. It covers TMM normalisation and filterByExpr; common, trended and tagwise dispersion estimation; negative binomial generalised linear models with likelihood-ratio and quasi-likelihood F-tests; tests against a fold-change threshold; and the classic exact test. Each function is available under its edgeR name and a Python name. pyedger was ported step by step from edgeR's compiled routines and from the limma and locfit code that edgeR calls, and it is versioned against edgeR 4.10.5. We compared pyedger 0.2.3 with edgeR 4.10.5 on eight tumour–normal cohorts of The Cancer Genome Atlas (4,972 samples) and on 60 small-sample draws, with both packages running the same default pipeline. Every differential-expression call of the quasi-likelihood, likelihood-ratio and exact tests was identical in every cohort and draw, and pathway analysis of both outputs on the eight cohorts gave the same significant pathways. P-values differed by at most 1.2×10^{-5} (relative difference), and with eight threads pyedger ran the full pipeline 3.4 to 3.5 times faster than edgeR on one thread. pyedger is the edgeR backend of the bulk differential expression and Milo differential abundance analyses in OmicVerse. It is available under the LGPL-3.0-or-later licence from PyPI and `https://github.com/omicverse/py-edgeR`.

Keywords edgeR, differential expression, RNA-seq, negative binomial, quasi-likelihood, Python, OmicVerse

Introduction

Differential expression analysis of RNA sequencing (RNA-seq) counts identifies the genes whose expression differs between conditions, and it is the starting point of most pathway and regulatory analyses of bulk and single-cell data[1]. edgeR is one of the standard Bioconductor packages for this task[2,3]. It models counts with the negative binomial (NB) distribution and shares information between genes to estimate the NB dispersion, which keeps tests reliable with few replicates[4,5]. Its generalised linear models (GLMs) handle arbitrary designs[6], its quasi-likelihood (QL) F-tests account for the uncertainty of the dispersion estimates[7], and its trimmed mean of M-values (TMM) normalisation corrects for differences in library composition[8]. Together these steps form the recommended edgeR pipeline[9].

Omics analysis increasingly takes place in Python. Scanpy[10] and the scverse ecosystem[11] provide data structures and methods for single-cell data, and OmicVerse[12] brings bulk and single-cell analyses into one framework. Count-based differential testing is needed throughout these workflows: for bulk RNA-seq, for pseudobulk profiles of cell populations and for the differential abundance of cell neighbourhoods. edgeR, however, is available only in R.

Python users therefore reach edgeR through rpy2[13], which embeds an R session in the Python process. This adds a second language runtime and its package environment, converts count matrices and result tables between the two languages, and exposes edgeR's options through a binding layer. A native implementation removes these costs, as PyDESeq2 has shown for DESeq2[14,15] and InMoose for several bulk transcriptomics methods[16]. We set a specific target for pyedger: an analysis in Python should return the same genes, the same statistics and the same downstream results as the same analysis in edgeR.

Here we present pyedger, a Python implementation of edgeR's NB GLM, quasi-likelihood and exact-test workflows. pyedger follows edgeR's function names, arguments and defaults (Table 1), needs no R installation, and runs its per-gene computations in parallel. We describe its implementation and compare it with edgeR 4.10.5 on eight tumour–normal cohorts of The Cancer Genome Atlas (TCGA) and on 60 small-sample draws.

© 2026 The Authors
Published at omicverse.org/articles/pyedger, version 1.

Table 1. Functions of edgeR and pyedger, with the largest difference from edgeR 4.10.5 in the TCGA benchmark (relative unless marked absolute; eight cohorts unless stated; logFC, log2 fold change) or, for functions outside the benchmark pipeline, the tolerance of their R-parity tests.

edgeR	pyedger	Largest difference from edgeR 4.10.5 on TCGA
`filterByExpr`	`filter_by_expr`	same genes kept in all cohorts and draws
`calcNormFactors` (TMM)	`calc_norm_factors`	normalisation factors 4.9×10^{-15} (all cohorts and draws)
`aveLogCPM`, `cpm`	`ave_log_cpm`, `cpm`	AveLogCPM 5×10^{-12}
`estimateDisp`	`estimate_disp`	common dispersion 1.1×10^{-12} and prior df 5.6×10^{-10} (all cohorts and draws); tagwise dispersion 8.6×10^{-10}
`glmQLFit`, `glmQLFTest`	`glm_ql_fit`, `glm_qlf_test`	P-value 1.2×10^{-5}; logFC 3.3×10^{-8} (absolute)
`glmFit`, `glmLRT`	`glm_fit`, `glm_lrt`	P-value 3.8×10^{-7}; logFC 9.7×10^{-13} (absolute)
`exactTest`	`exact_test`	P-value 5.1×10^{-8}; logFC 9.7×10^{-13} (absolute)
Outside the benchmark pipeline: tolerance of the R-parity tests		
`glmTreat`	`glm_treat`	P-value within 10^{-6} (GLM and legacy QL fits) or 10^{-4} (default QL fits)
`estimateGLMCommonDisp`	`estimate_glm_common_disp`	common dispersion within 10^{-4}

Implementation

Statistical model

edgeR models the count of a gene in a sample as NB[6] with mean μ and dispersion ϕ, so that the variance is $\mu + \phi\mu^2$. The mean follows a log-linear model of the design, with the log of the TMM-normalised library size as offset[8]. Dispersions are estimated in three layers[17]: a common value, a smooth trend against average abundance[18], and gene-wise (tagwise) values shrunk towards the trend by weighted likelihood empirical Bayes[4], with prior degrees of freedom estimated from the data[19,20]. With a design, the likelihood is the Cox–Reid adjusted profile likelihood[6]; without one, it is the conditional likelihood of quantile-adjusted pseudo-counts, which also feed the exact test[5]. Genes are tested with GLM likelihood-ratio tests[6], with QL F-tests whose QL dispersions are moderated by an empirical Bayes F-distribution fit[7,19], and against a fold-change threshold[21]. Since edgeR 4.2 the default QL method adjusts deviances and residual degrees of freedom for the bias of the NB unit deviance at small counts[3]. False discovery rates (FDRs) follow Benjamini and Hochberg[22].

Features and code structure

pyedger 0.2.3 implements the edgeR functions of the standard workflows (Table 1). For normalisation and filtering it provides the `DGEList` container, `calcNormFactors` (TMM, TMMwsp, RLE and upper quartile), `filterByExpr`, `cpm` and `aveLogCPM`. For dispersions it provides `estimateDisp`, with and without a design, and the `estimateGLM*Disp` functions, including the observation-weighted robust dispersion[23]. For testing it provides the likelihood-ratio test (`glmFit`, `glmLRT`), the QL F-test (`glmQLFit`, `glmQLFTest`), the test against a fold-change threshold (`glmTreat`) and the exact test (`exactTest`), with `topTags` and `decideTests` for the results. Every function is exported under its edgeR name and a Python name (`estimateDisp` and `estimate_disp`), so an edgeR script translates line by line. Results are dataclasses that mirror edgeR's S4 classes (`DGEGLM`, `DGELRT`, `DGEExact`). Their tables are pandas DataFrames[24] with edgeR's column names (`logFC`, `logCPM`, `F`, `PValue`, `FDR`). The numerical code is written with NumPy[25] and SciPy[26], vectorised over genes, with numba[27] for the compiled per-gene kernels.

Numerical parity with edgeR

pyedger ports edgeR's computations step by step, with the same starting values, damping, tolerances and floors. The ported code covers edgeR's compiled routines for the GLM fits, the NB unit deviance, the adjusted profile likelihood and the bias adjustments of the default QL method. It also covers the limma 3.68.5 functions that edgeR calls[28], such as `squeezeVar`, and the locfit 1.5-9.12 code behind the dispersion trend[18]. R routines such as Brent's optimiser, the FMM cubic spline and `lowess` are reproduced where edgeR depends on them.

One detail matters on large cohorts. In `q2qnbinom`, which turns counts into pseudo-counts for the classic dispersion estimate and the exact test, tail probabilities are evaluated on the log scale, as R's `pgamma(log.p = TRUE)` does; pseudo-counts therefore stay finite for strongly differentially expressed genes.

The test suite has 42 tests, 28 of which compare pyedger with references from edgeR 4.10.5 and limma 3.68.5 (R 4.5.3), generated by `tests/data/make_reference.R`. They cover a simulated paired design with all-zero library pairs, OmicVerse's Milo design, limma hyperparameters, extreme `q2qnbinom` inputs and TCGA genes for the exact test. pyedger 0.2.x tracks edgeR 4.10.5 (Bioconductor 3.23) and uses the defaults of edgeR 4.10.

Performance

On TCGA-BRCA, the largest cohort, `estimateDisp` takes more than half of edgeR's run time (Supplementary methods), because it fits the GLM of every gene at each of 21 dispersion grid points. pyedger fits all genes of a grid point at once, warm-starts each fit from the previous grid point and computes once what the grid points share, such as $\log\Gamma(y+1)$ and the group sub-matrices of one-way designs. The per-gene loops of the Levenberg and one-group GLM fits and of the QL adjustments are numba kernels that run in parallel over genes. The vectorised NumPy code of the one-way fits of two-group designs, the adjusted profile likelihood, the NB deviance and `q2qnbinom` runs on a thread pool over blocks of genes. No computation combines genes across blocks or threads, so results do not depend on the number of threads.

Use in OmicVerse

pyedger is the edgeR backend of OmicVerse, which installs it as a dependency and exposes it as `omicverse.external.pyedger`.

In `ov.bulk.pyDEG`, `deg_analysis` with `method="edger"` runs `calcNormFactors`, `estimateDisp`, `glmQLFit` and `glmQLFTest`. With `method="edgepy"` it runs `glmFit` and `glmLRT` with the tagwise dispersions, and with `method="limma"` it takes TMM effective library sizes from pyedger for voom[29]. `ov.single.Milo` tests the differential abundance of cell neighbourhoods[30]: its `da_nhoods` method with `solver="edger"` calls `estimateDisp`, `glmQLFit` and `glmQLFTest`. `ov.single.DCT` with `method="milopy"` runs this test through `da_nhoods`. The spike-in ChIP-seq function `ov.bulk.spikein_chip_analysis` compares paired samples with pyedger's robust dispersion[23] and likelihood-ratio test, or with robust QL fits.

Comparison with edgeR on TCGA tumour–normal cohorts

Data and design

We compared pyedger 0.2.3 with edgeR 4.10.5 on eight TCGA cohorts[31]: BRCA, COAD, KIRC, LIHC, LUAD, LUSC, PRAD and THCA. STAR gene counts came from the UCSC Xena GDC hub[32], and each comparison was primary tumour versus solid tissue normal. The cohorts have 421 to 1,208 samples (4,972 in total). Because edgeR is often used with few replicates, we also took 60 small-sample draws from TCGA-LUAD: 20 random draws each of 3, 5 and 10 tumours versus as many normals (Supplementary methods).

Both packages ran the default edgeR 4.10 pipeline[9] on the eight cohorts and the 60 draws (68 tasks): `filterByExpr`, TMM normalisation and `estimateDisp` with the design `~group`. Genes were then tested with the QL F-test, the GLM likelihood-ratio test and the classic exact test, with Benjamini–Hochberg FDRs. A gene was called up or down when FDR ≤ 0.05 and its absolute log2 fold change (logFC) was at least 1. We compared four criteria: the calls of each test; a pathway analysis run on both outputs; numerical agreement; and the wall time of the full pipeline, with one thread for both packages and eight threads for pyedger. Numerical agreement covered the relative differences of the per-gene AveLogCPM, tagwise dispersion, logFC and P-values and of the normalisation factors, common dispersion and prior degrees of freedom.

Differential-expression calls and pathway analysis

`filterByExpr` kept the same genes in all 68 tasks. On the eight cohorts, edgeR's QL F-test made 61,578 differential-expression calls (summed over cohorts), and pyedger made the same call for every gene (Fig. 1a). The likelihood-ratio and exact tests also had no discordant calls, and in every test each gene fell on the same side of the FDR threshold in both packages. In the 60 small-sample draws the three tests again gave identical calls. QL P-values of the two packages lay on the identity line in every cohort (Fig. S1).

For the downstream analysis, fgsea[33,34] tested the MSigDB Hallmark gene sets[35] in each cohort, with genes ranked by the signed square root of the QL F-statistic and the same seed for both packages. Of the 400 cohort–pathway pairs, 257 were significant with edgeR statistics, and pyedger statistics gave the same significance for all 400 pairs. A second fgsea seed on edgeR statistics changed the significance of eight pairs. Normalised enrichment scores differed by at most 2.5×10^{-8} between the packages and by up to 0.081 between seeds (Fig. 1b).

Numerical agreement and speed

Outside the QL fit, the largest relative differences of normalisation factors, dispersions and P-values between pyedger and edgeR ranged from 4.9×10^{-15} for the normalisation factors to 3.8×10^{-7} for the likelihood-ratio P-values (Table 1, Fig. 1c). QL F-test P-values agreed within 1.2×10^{-5}, and in the small-sample draws the largest P-value difference over the three tests was 1.3×10^{-6} (Fig. 1e).

On one thread pyedger ran the full pipeline 1.1 to 1.3 times faster than edgeR, and on eight threads 3.4 to 3.5 times faster (Fig. 1d). On TCGA-BRCA the pipeline took 351 s in edgeR, 277 s in pyedger on one thread and 102 s on eight threads. pyedger on eight threads returned results bit-identical to its single-threaded run on all eight cohorts.

Conclusion and future perspectives

pyedger brings edgeR's NB GLM, quasi-likelihood and exact-test workflows to Python with results that match edgeR 4.10.5. On 4,972 TCGA samples and 60 small-sample draws every differential-expression call was the same, every pathway call on the eight cohorts was the same, and P-values agreed within 1.2×10^{-5}. With eight threads the pipeline ran 3.4 to 3.5 times faster than edgeR on one thread. Because function names, arguments and defaults follow edgeR, an edgeR analysis moves to Python line by line, and its results can be checked against an existing R analysis. Through OmicVerse, the same implementation serves bulk differential expression, Milo differential abundance and spike-in ChIP-seq comparisons.

Future work includes the parts of edgeR that pyedger 0.2.3 does not yet cover: the gene-set, splicing and data-import functions; the binned trend methods of `estimateGLMTrendedDisp`; dispersion methods other than Cox–Reid in `estimateGLMCommonDisp`; `topTags` adjustment methods other than Benjamini–Hochberg; and sparse count matrices. We also plan to replace pyedger's `estimateTagwiseDisp`, which shrinks moment estimates towards the common dispersion, with a port of edgeR's weighted-likelihood estimator. Further R-parity tests will cover the robust dispersion, the alternative trend methods of `estimateDisp` and the exact-test rejection regions other than the default. New pyedger releases will follow edgeR and limma releases on Bioconductor, with the R references regenerated for each release.

Data and code availability

pyedger is free software under the GNU Lesser General Public License v3.0 or later. It is installed from PyPI with `pip install pyedger` and developed at github.com/omicverse/py-edgeR; this report describes version 0.2.3 (Git tag `v0.2.3`). The benchmark scripts, figures, per-task tables and the numbers quoted here are in the `report/` directory of the repository, whose README gives the commands that reproduce them. TCGA STAR gene counts (`TCGA-BRCA.star_counts.tsv.gz` and the corresponding files of the other seven cohorts) and the GENCODE v36 gene probe map[36] were downloaded from the UCSC Xena GDC hub at `https://gdc-hub.s3.us-east-1.amazonaws.com/download/`. The MSigDB Hallmark 2020 gene sets were downloaded from the Enrichr library `MSigDB_Hallmark_2020` (maayanlab.cloud/Enrichr). The benchmark used R 4.5 with edgeR 4.10.5, limma 3.68.5 and

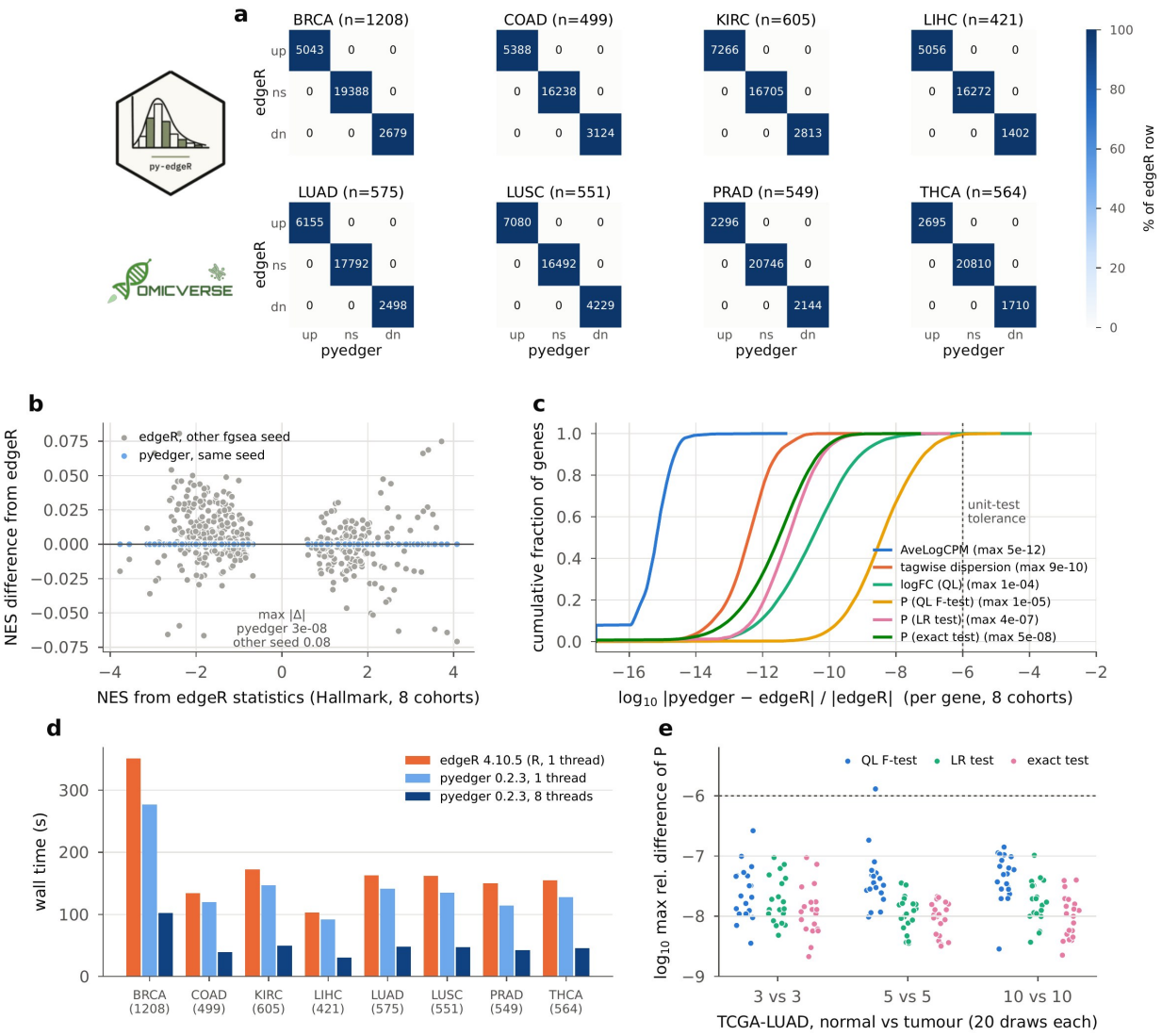


Figure 1. pyedger 0.2.3 versus edgeR 4.10.5 on eight TCGA tumour–normal cohorts and 60 small-sample draws.

(a) QL F-test calls (Benjamini–Hochberg FDR ≤ 0.05, absolute log2 fold change of at least 1) in each cohort: rows, edgeR calls; columns, pyedger calls (up, not significant (ns), down (dn)); cells, numbers of genes, coloured by the percentage of the edgeR row; n, number of samples. **(b)** fgsea on the Hallmark gene sets: difference from the normalised enrichment score (NES) of edgeR statistics with seed 1 (y axis) against that NES (x axis), for pyedger statistics with seed 1 (blue) and edgeR statistics with seed 2 (grey); 400 cohort–pathway pairs each; annotation, largest absolute difference. **(c)** Cumulative distributions of per-gene relative differences ($\log_{10}$) over the 206,021 genes of the eight cohorts for AveLogCPM, tagwise dispersion, QL logFC and the P-values of the QL F-test, likelihood-ratio test and exact test; the panel legend gives each maximum; values below 10^{-17}, including zeros, are plotted at -17; dashed line, 10^{-6}, the default tolerance of the R-parity tests (values that depend on the NB dispersion of the QL fit are tested at 10^{-4}, which is not drawn). **(d)** Wall time of the full pipeline per cohort (samples in brackets) for edgeR on one thread (orange) and pyedger on one (light blue) and eight threads (dark blue). **(e)** Largest relative P-value difference ($\log_{10}$) in each TCGA-LUAD draw (20 draws each of 3, 5 and 10 tumours versus as many normals) for the QL F-test (blue), likelihood-ratio test (green) and exact test (pink); dashed line, 10^{-6}, the default tolerance of the R-parity tests.

fgsea, and pyedger 0.2.3, which requires Python $\geq$ 3.9, NumPy $\geq$ 1.23, SciPy $\geq$ 1.10, pandas $\geq$ 1.5 and numba $\geq$ 0.57.

Use of AI tools

Claude (Anthropic) was used as a coding assistant in the development of pyedger. Ten of the 16 commits in the repository are co-authored by Claude; they cover the package code, the R-parity tests and reference scripts, the benchmark scripts and the documentation. Claude also drafted this report from the repository, the benchmark outputs and the cited literature. The authors reviewed and edited the code and the text and take full responsibility for the content.

Conflicts of interest

Z.Z. is affiliated with OmicVerse, Inc. M.X. and H.D. declare no conflicts of interest.

References

[1] Van den Berge K, Hembach KM, Soneson C, et al. (2019). RNA Sequencing Data: Hitchhiker's Guide to Expression Analysis. Annual Review of Biomedical Data Science 2, 139-173. 10.1146/annurev-biodatasci-072018-021255

[2] Robinson MD, McCarthy DJ, Smyth GK (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. Bioinformatics 26, 139-140. 10.1093/bioinformatics/btp616

[3] Chen Y, Chen L, Lun ATL, Baldoni PL, Smyth GK (2025). edgeR v4: powerful differential analysis of sequencing data with expanded functionality and improved support for small counts and larger datasets. Nucleic Acids Research 53, gkaf018. 10.1093/nar/gkaf018

[4] Robinson MD, Smyth GK (2007). Moderated statistical tests for assessing differences in tag abundance. Bioinformatics 23, 2881-2887. 10.1093/bioinformatics/btm453

[5] Robinson MD, Smyth GK (2007). Small-sample estimation of negative binomial dispersion, with applications to SAGE data. Biostatistics 9, 321-332. 10.1093/biostatistics/kxm030

[6] McCarthy DJ, Chen Y, Smyth GK (2012). Differential expression analysis of multifactor RNA-Seq experiments with respect to biological variation. Nucleic Acids Research 40, 4288-4297. 10.1093/nar/gks042

[7] Lund SP, Nettleton D, McCarthy DJ, Smyth GK (2012). Detecting Differential Expression in RNA-sequence Data Using Quasi-likelihood with Shrunken Dispersion Estimates. Statistical Applications in Genetics and Molecular Biology 11. 10.1515/1544-6115.1826

[8] Robinson MD, Oshlack A (2010). A scaling normalization method for differential expression analysis of RNA-seq data. Genome Biology 11, R25. 10.1186/gb-2010-11-3-r25

[9] Chen Y, Lun ATL, Smyth GK (2016). From reads to genes to pathways: differential expression analysis of RNA-Seq experiments using Rsubread and the edgeR quasi-likelihood pipeline. F1000Research 5, 1438. 10.12688/f1000research.8987.2

[10] Wolf FA, Angerer P, Theis FJ (2018). SCANPY: large-scale single-cell gene expression data analysis. Genome Biology 19, 15. 10.1186/s13059-017-1382-0

[11] Virshup I, Bredikhin D, Heumos L, et al. (2023). The scverse project provides a computational ecosystem for single-cell omics data analysis. Nature Biotechnology 41, 604-606. 10.1038/s41587-023-01733-8

[12] Zeng Z, Ma Y, Hu L, et al. (2024). OmicVerse: a framework for bridging and deepening insights across bulk and single-cell sequencing. Nature Communications 15, 5983. 10.1038/s41467-024-50194-3

[13] Gautier L. rpy2: Python interface to the R language. https://rpy2.github.io

[14] Muzellec B, Teleńczuk M, Cabeli V, Andreux M (2023). PyDESeq2: a python package for bulk RNA-seq differential expression analysis. Bioinformatics 39, btad547. 10.1093/bioinformatics/btad547

[15] Love MI, Huber W, Anders S (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology 15, 550. 10.1186/s13059-014-0550-8

[16] Colange M, Appé G, Meunier L, et al. (2025). Bridging the gap between R and Python in bulk transcriptomic data analysis with InMoose. Scientific Reports 15, 18104. 10.1038/s41598-025-03376-y

[17] Chen Y, Lun ATL, Smyth GK (2014). Differential Expression Analysis of Complex RNA-seq Experiments Using edgeR. Statistical Analysis of Next Generation Sequencing Data, 51-74. 10.1007/978-3-319-07212-8_3

[18] Loader C (1999). Local Regression and Likelihood. Springer New York. 10.1007/b98858

[19] Smyth GK (2004). Linear Models and Empirical Bayes Methods for Assessing Differential Expression in Microarray Experiments. Statistical Applications in Genetics and Molecular Biology 3, 1-25. 10.2202/1544-6115.1027

[20] Phipson B, Lee S, Majewski IJ, Alexander WS, Smyth GK (2016). Robust hyperparameter estimation protects against hypervariable genes and improves power to detect differential expression. The Annals of Applied Statistics 10, 946-963. 10.1214/16-aoas920

[21] McCarthy DJ, Smyth GK (2009). Testing significance relative to a fold-change threshold is a TREAT. Bioinformatics 25, 765-771. 10.1093/bioinformatics/btp053

[22] Benjamini Y, Hochberg Y (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. Journal of the Royal Statistical Society Series B: Statistical Methodology 57, 289-300. 10.1111/j.2517-6161.1995.tb02031.x

[23] Zhou X, Lindsay H, Robinson MD (2014). Robustly detecting differential expression in RNA sequencing data using observation weights. Nucleic Acids Research 42, e91-e91. 10.1093/nar/gku310

[24] McKinney W (2010). Data Structures for Statistical Computing in Python. Proceedings of the Python in Science Conference, 56-61. 10.25080/majora-92bf1922-00a

[25] Harris CR, Millman KJ, van der Walt SJ, et al. (2020). Array programming with NumPy. Nature 585, 357-362. 10.1038/s41586-020-2649-2

[26] Virtanen P, Gommers R, Oliphant TE, et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 17, 261-272. 10.1038/s41592-019-0686-2

[27] Lam SK, Pitrou A, Seibert S (2015). Numba: a LLVM-based Python JIT compiler. Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 1-6. 10.1145/2833157.2833162

[28] Ritchie ME, Phipson B, Wu D, et al. (2015). limma powers differential expression analyses for RNA-sequencing and microarray studies. Nucleic Acids Research 43, e47-e47. 10.1093/nar/gkv007

[29] Law CW, Chen Y, Shi W, Smyth GK (2014). voom: precision weights unlock linear model analysis tools for RNA-seq read counts. Genome Biology 15, R29. 10.1186/gb-2014-15-2-r29

[30] Dann E, Henderson NC, Teichmann SA, Morgan MD, Marioni JC (2022). Differential abundance testing on single-cell data using k-nearest neighbor graphs. Nature Biotechnology 40, 245-253. 10.1038/s41587-021-01033-z

[31] The Cancer Genome Atlas Research Network, Weinstein JN, Collisson EA, et al. (2013). The Cancer Genome Atlas Pan-Cancer analysis project. Nature Genetics 45, 1113-1120. 10.1038/ng.2764

[32] Goldman MJ, Craft B, Hastie M, et al. (2020). Visualizing and interpreting cancer genomics data via the Xena platform. Nature Biotechnology 38, 675-678. 10.1038/s41587-020-0546-8

[33] Korotkevich G, Sukhov V, Budin N, Shpak B, Artyomov MN, Sergushichev A (2016). Fast gene set enrichment analysis. bioRxiv. 10.1101/060012

[34] Subramanian A, Tamayo P, Mootha VK, et al. (2005). Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles. Proceedings of the National Academy of Sciences 102, 15545-15550. 10.1073/pnas.0506580102

[35] Liberzon A, Birger C, Thorvaldsdóttir H, Ghandi M, Mesirov JP, Tamayo P (2015). The Molecular Signatures Database Hallmark Gene Set Collection. Cell Systems 1, 417-425. 10.1016/j.cels.2015.12.004

[36] Frankish A, Diekhans M, Jungreis I, et al. (2021). GENCODE 2021. Nucleic Acids Research 49, D916-D923. 10.1093/nar/gkaa1087

Supplementary information

Supplementary methods

Data processing. UCSC Xena stores the GDC STAR counts as $\log_2(\text{count}+1)$, and counts were recovered as $\text{round}(2^x-1)$. Samples whose recovered values were not integers within 10^{-6}, which are averages of replicate aliquots, were removed. Genes with identifiers ending in `_PAR_Y` were removed. Only primary tumours (sample type 01) and solid tissue normals (sample type 11) were kept, with one aliquot per participant and sample type (the first after sorting the barcodes). Genes with a zero total count in the cohort were removed. Normal tissue is the reference level, so log fold changes are tumour versus normal. `filterByExpr` kept 22,730 to 27,801 genes per cohort.

Small-sample draws. Each draw took n tumours and n normals from the processed TCGA-LUAD cohort, without replacement within the draw, for n = 3, 5 and 10, with 20 draws per size. Draws are independent, so a sample can appear in several draws. All draws came from one generator, `numpy.random.default_rng(20261109)`, used in the order n = 3, 5, 10. Genes with a zero total over the drawn samples were removed, and `filterByExpr` then kept 18,455 to 21,161 genes per draw.

Pipeline and calls. Both packages were run with default arguments: `DGEList` with the group labels; `filterByExpr`; library sizes recomputed after filtering (R's `keep.lib.sizes = FALSE`); TMM normalisation; the design `~group`; `estimateDisp(y, design)`; `glmQLFit` and `glmQLFTest` on the group coefficient (index 1 in pyedger, 2 in R); `glmFit` and `glmLRT` on the same coefficient; and `exactTest`. FDRs were computed with R's `p.adjust(method = "BH")` and SciPy's `false_discovery_control(method="bh")`. A gene was called up for FDR ≤ 0.05 and logFC ≥ 1, down for FDR ≤ 0.05 and logFC ≤ -1, and not significant otherwise. Relative differences are $|a-b|/\max(|a|, 10^{-250})$, with a the edgeR value, so P-values below 10^{-250} count as zero; logFC differences are also given as absolute differences. The single-threaded pyedger run was compared with edgeR, and the eight-thread run with the single-threaded run.

Pathway analysis. fgsea was run on the QL F-test results of each cohort with the MSigDB Hallmark 2020 gene sets (400 cohort–pathway pairs in total). Genes were ranked by $\text{sign}(\text{logFC})\sqrt{\max(F,0)}$. Ensembl identifiers without version were mapped to gene symbols with the GENCODE v36 probe map[36]; for a duplicated symbol the gene with the largest absolute statistic was kept. The parameters were `minSize = 15`, `maxSize = 500`, `eps = 0` and `nproc = 1`. edgeR and pyedger statistics were run with `set.seed(1)`, and edgeR statistics once more with `set.seed(2)`. A pathway was significant at adjusted P ≤ 0.05.

Run time. Run times are wall-clock times of the five pipeline stages (filtering and normalisation, `estimateDisp`, QL fit and test, GLM fit and likelihood-ratio test, exact test), measured with `proc.time()` in R and `time.perf_counter()` in Python. Reading the counts, the FDR adjustment and writing the results are not included. Each pyedger process first ran the pipeline once, untimed, on a small task, so that numba compilation and cache loading were not timed. The benchmark driver times each task and configuration once by default (`REPS=1`), and the README command keeps that default. BLAS was pinned to one thread (`OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS` and `MKL_NUM_THREADS` set to 1). edgeR and then single-threaded pyedger each ran as nine parallel processes, one per cohort and one for the 60 draws. pyedger on eight threads then ran as a single process over the eight cohorts. On TCGA-BRCA the stages took, in edgeR, pyedger on one thread and pyedger on eight threads: filtering and normalisation, 23.0, 16.7 and 16.7 s; `estimateDisp`, 199.9, 108.3 and 35.9 s; QL fit and test, 44.1, 54.0 and 24.4 s; GLM fit and likelihood-ratio test, 14.8, 18.4 and 9.1 s; exact test, 69.4, 79.7 and 16.1 s.

Implementation details for bit-level agreement. The NB unit deviance follows edgeR's C code: 10^{-8} is added to counts and means, dispersions below 10^{-4} use the Poisson branch (keeping C's integer division `2/3`), $\mu\phi > 10^6$ uses the gamma branch, and unit deviances are floored at zero. In the numba kernels and in the bias adjustments of the default QL method, the logarithms of the unit deviance go through the C library (`math.log`), whose results differ from NumPy's vectorised `np.log` by one unit in the last place on about 0.4% of arguments. The Cox–Reid adjustment is the log-determinant of an LDL^T factorisation with zero pivots floored at $\log(10^{-10})$, and designs are reduced to estimable columns with R's `qr()` rank rule (tolerance 10^{-7}). Dispersion maxima on the grid are found with R's `maximizeInterpolant` (FMM cubic spline). R's `Brent_fmin` gives the classic common dispersion and `dispCoxReid`, which `glmQLFit` uses for the NB dispersion; the latter optimises $\phi^{1/4}$ on $[0, 4^{1/4}]$ with tolerance 10^{-5}. TMM takes log2 through the C library and ranks ties by their average. In `q2qnbinom`, `pgamma` and `qgamma` are evaluated on the log scale with a series for the lower tail and a Lentz continued fraction for the upper tail, and the result averages the normal and gamma quantile maps. The exact test uses the beta approximation when both group sums exceed 900. Its double-tail sum runs over the same range as R's `a:b`, including when negative pseudo-counts make a group sum exceed the total. The QL weights use the Chebyshev coefficient tables of edgeR's `ql_weights.c`, and the QL prior is a lowess fit ($f = 0.5$, three iterations) followed by the 90% type-7 quantile.

Notes for users coming from R

1. Versions and defaults. pyedger 0.2.3 tracks edgeR 4.10.5 and limma 3.68.5, and its defaults are those of edgeR 4.10.
   - `glmQLFit` uses the bias-adjusted QL method by default (`legacy=False`, as in edgeR 4.2 and later); `legacy=True` gives the classic method.
   - `estimateDisp` estimates the prior degrees of freedom (`prior_df=None`) and chooses its span with `chooseLowessSpan`; `legacy_span=True` gives the span of edgeR 4.6 and earlier.
   - `glmQLFit(legacy=False)` does not use dispersions stored in the `DGEList` and caps given dispersions at 4, as edgeR 4.10 does.

- `estimateTagwiseDisp` shrinks moment estimates towards the common dispersion with `prior_df=10`, not edgeR's weighted likelihood; `estimateDisp` gives edgeR's tagwise dispersions.

2. Indices. Coefficients are 0-based: `coef=1` is R's `coef = 2`, and coefficient names are also accepted. An integer `coef` equal to the number of design columns is read as R's last coefficient. `exactTest(pair=...)` takes level labels or 1-based positions, as in R.

3. Subsetting. `y[keep]` keeps the original library sizes, as R does by default; `y.subset(keep, keep_lib_sizes=False)` corresponds to `keep.lib.sizes = FALSE`.

4. Arguments. `estimateDisp(y, design)` takes the design as its second positional argument, as in R. The `dispersion` argument of `exactTest` accepts a numeric vector.

5. Groups and designs. A `pandas.Categorical` group keeps its level order, and the first level is the reference; plain lists are sorted, as R's `factor()` does. Design columns built from the group are named by the level (`treatment`), not R's `grouptreatment`, and NumPy designs get the names `coef_0`, `coef_1` and so on.

6. Scales and tables. logFC is on the log2 scale and model coefficients on the natural-log scale, as in R. `topTags` returns a pandas DataFrame with Benjamini–Hochberg FDRs.

7. Threads. The numba kernels follow `NUMBA_NUM_THREADS`; the thread pool follows `PYEDGER_NUM_THREADS` and otherwise numba's thread count. Results do not depend on either.

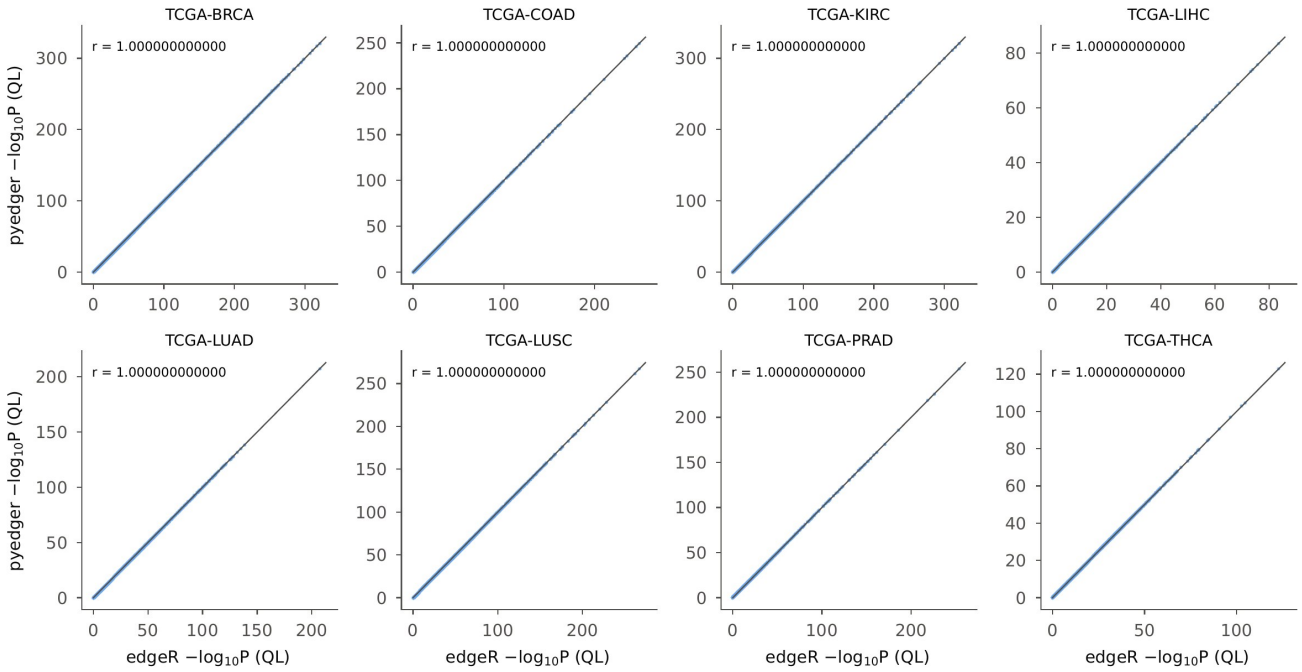


Figure S1. QL F-test P-values of pyedger 0.2.3 versus edgeR 4.10.5 in the eight TCGA cohorts.

Each panel plots $-\log_{10} P$ of every gene kept by `filterByExpr` in one cohort, pyedger against edgeR, with the identity line (grey); P-values are floored at 10^{-320}; r is the Pearson correlation of $-\log_{10} P$, printed to 12 decimals.